

// Contents

0. 프로젝트 한눈에 보기
1. 아키텍처 전체 흐름
2. 크로스플랫폼 코어
3. 보안 설계
4. 통신 프로토콜
5. 데이터 저장
6. 수집 지표 구현 상세
7. 성능/권한 설계
8. APM_Console 아키텍처
9. 발견한 버그 (7건)
10. 테스트 전략
11. 예상 질문 통합
12. 용어 미니 사전
13. 라이브 데모 순서

PROJECT_TECHNICAL_REVIEW

APM_Agent / APM_Console 프로젝트 복습 + 면접 준비용 기술문서 — HTML 버전(시각 도표 추가). "이미 다 안다"가 아니라 "설명이 필요할 때 바로 펼쳐볼 수 있다"를 목표로 작성했다. 원본은 [Docs/PROJECT_TECHNICAL_REVIEW.md](#).

각 섹션의 Q&A 박스는 실제로 나올 법한 질문 + 답변 포인트.

// Overview

0. 프로젝트 한눈에 보기

이게 뭔가

[APM_Agent](#) (C++)와 [APM_Console](#) (.NET)로 이루어진 경량 모니터링(APM/오피저버빌리티) 시스템. 세 개의 프로세스가 순서대로 연결된다:

- **Agent:** CPU/메모리/디스크/네트워크/TCP 연결 품질을 주기적으로 수집해서 Collector로 전송.
- **Collector:** Agent로부터 받은 데이터를 SQLite에 저장하고, 동시에 APM_Console로도 전송(재전송).
- **APM_Console:** Collector가 보낸 데이터를 받아서 자체 DB에 저장하고, 웹 브라우저에서 실시간 그래프/표로 보여줌.

왜 이 프로젝트를 만들었나

원래 이 저장소(`gw2-cross`)는 IOCP 기반 게임 서버를 크로스플랫폼으로 포팅하는 프로젝트로 시작했는데, 목표가 "APM/옵저버빌리티 개발자 포지션 지원용 포트폴리오"로 재정의되면서, 기존에 만들던 크로스플랫폼 네트워크 코어(`GW2_CrossPlatformCore`) 위에 이 APM 시스템을 새로 얹었다.

▲ 네이밍 원칙 — 면접에서 반드시 지킬 것

내부 문서(WORK_STATUS.md 등)는 "게임 서버"라는 표현을 그대로 쓰지만, **대외적으로(면접 포함) 설명할 때는 절대 "게임"이라는 단어를 쓰면 안 된다.**

- "게임 서버" → "비동기 I/O 기반 다중 클라이언트 TCP 서버"
- "게임 클라이언트 접속 처리" → "다중 에이전트 동시 접속 처리"
- 만들게 된 계기 → "네트워크 프로그래밍과 동시성 제어를 깊게 익히고 싶어서 직접 구현했다"

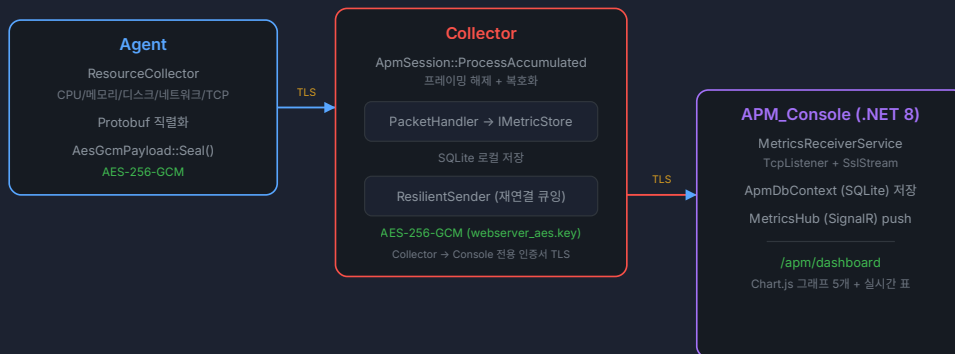
기술 스택 요약

영역	기술
네트워크 코어(C++)	C++20, Standalone Asio(Boost 미사용)
보안(현재)	TLS(OpenSSL) + AES-256-GCM(AEAD)
보안(과거 구현 경험)	ARIA-256-CBC + HMAC-SHA256(Encrypt-then-MAC)
직렬화	Protobuf
저장(Agent 쪽)	SQLite / PostgreSQL-TimescaleDB(컴파일 타임 선택)
웹 대시보드	.NET 8, ASP.NET Core MVC + Razor, EF Core, SignalR
플러그인 구조	Razor Class Library + AssemblyLoadContext
테스트	GoogleTest(C++) 9개, xUnit(.NET) 8개

// Architecture

1. 아키텍처 전체 흐름

// 전체 파이프라인 - 3개 프로세스, 2개의 암호화 구간



Collector는 "저장"과 "APM_Console로 재전송"을 동시에, 서로 독립적으로 수행한다

Agent↔Collector, Collector↔Console 모두 AES-256-GCM — 단, 키는 구간별로 분리
(agent_collector_aes.key / webserver_aes.key)

핵심은 **Collector가 두 가지 역할을 동시에 한다는 것**: (1) Agent로부터 받은 데이터를 로컬 SQLite에 저장하는 "수집 서버" 역할과, (2) 저장한 데이터를 다시 APM_Console로 넘겨주는 "중계자" 역할. 이 둘은 서로 독립적이다 — 저장이 실패해도 전송은 별개로 진행되고, 전송(WebServer 연결)이 끊겨도 로컬 저장은 계속된다.

// GW2_CrossPlatformCore를 왜 공유하나

이 프로젝트의 네트워크 코어(`GW2_CrossPlatformCore`)는 원래 게임 서버 포팅용으로 만든 것인데, `APM_Agent`도 이걸 그대로 가져다 쓴다. 이유는 단순하다 — **비동기 I/O 코어 자체는 도메인(게임이나 모니터링이나)과 무관하기 때문이다**.

`asio::io_context` 기반 이벤트 루프, TLS 소켓 래핑, 스레드 풀 관리 같은 건 "TCP로 여러 클라이언트를 비동기로 상대한다"는 문제의 해법이지 게임 전용 로직이 아니다.

```
// CMakeLists.txt
add_subdirectory(${CMAKE_CURRENT_SOURCE_DIR}/../GW2_Server/GW2_CrossPlatformCore ...
add_subdirectory(Common) // APM_Agent 전용 코드
add_subdirectory(Storage) // APM_Agent 전용 코드
```

Q. 왜 게임 서버 코드베이스에서 APM 에이전트를 만들었나요?

→ 기존에 크로스플랫폼 비동기 네트워크 코어를 만들어둔 게 있었고, 그 코어 자체는 특정 도메인에 종속되지 않는 범용 인프라라 APM 에이전트에도 그대로 재사용할 수 있었습니다. 코어(네트워크 계층)와 애플리케이션 로직(무엇을 주고받을지)을 분리해서 설계했기 때문에 가능했던 재사용입니다.

```
// Cross-Platform Core
```

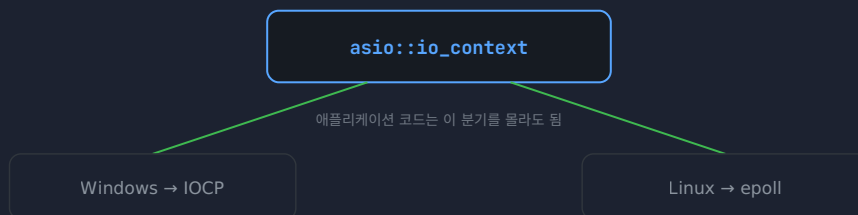
2. 크로스플랫폼 코어 – Standalone Asio

원본(`GW2_Server/GW2_ServerCore`)은 Windows 전용 **IOCP**(I/O Completion Port)로 짜여 있었다. Windows API에 강하게 결합돼 있어서 Linux 등 다른 OS에서는 아예 컴파일조차 안 된다.

Linux의 비동기 I/O 메커니즘은 **epoll**이다. IOCP와 epoll은 개념(완료 통지 vs 준비 통지)부터 다르고, 이 둘을 직접 추상화하는 레이어를 처음부터 만드는 건 상당한 공수다.

Standalone Asio(Boost 전체가 아니라 Asio 라이브러리만 떼어 쓰는 버전)는 이미 이 추상화를 제공한다.

```
// OS별 비동기 I/O 추상화
```



설계 원칙: 비동기 I/O·멀티스레드 처리 "구조" 자체(이벤트 루프, 콜백 기반 처리)는 기존 설계를 유지하고, 그 아래 OS별 구현 디테일만 Asio로 교체했다.

검증: Linux에서 빌드+실행까지 검증(포트 리스닝, 무크래시). 게임 콘텐츠 레이어까지 없어서 실제로 end-to-end로 동작함을 확인한 이력도 있음(포트 7777 리스닝, SIGTERM 정상 종료).

Windows는 왜 재검증 안 했나 — 크로스플랫폼 포팅의 원래 동기 자체가 "Linux에서도 돌아가게 하기"였다. Windows는 이미 검증된 원본 IOCP 버전이 있으니, 굳이 재검증할 필요를 못 느꼈다. "다 못 했다"가 아니라 **의도적으로 스코프를 좁힌 결정**이다.

Q. Standalone Asio가 정확히 뭔가요? Boost.Asio랑 다른가요?

→ Boost.Asio는 Boost 라이브러리 전체의 일부로 배포되는데, Standalone Asio는 Boost 의존성 없이 Asio 라이브러리 코드만 떼어낸 버전입니다. Boost 전체를 빌드에 끌어오지 않아도 되는 게 장점이고, API는 사실상 동일합니다.

Q. 크로스플랫폼이라면서 왜 Windows는 검증 안 했나요?

→ 포팅 동기 자체가 'Linux에서도 돌아가는 것'이었고, Windows는 이미 검증된 원본이 있어서 우선순위상 뒤로 미룬 의도적 스코프 결정이었습니다. Asio 자체가 Windows에서 IOCP를 쓴다는 건 라이브러리 차원에서 보장되는 부분이라, 재검증의 한계효용이 낮다고 판단했습니다.

// Security

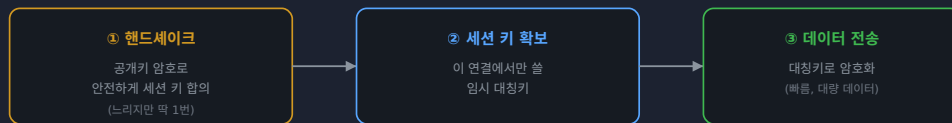
3. 보안 설계

이 프로젝트에서 가장 공들인 부분이자, 실제로 설계를 한 번 뒤집은(ARIA→AES) 이력이 있는 영역이다.

3-1. TLS 계층 – 기반이 되는 전송 보안

TLS는 공개키(비대칭) 암호와 대칭키 암호를 함께 쓰는 하이브리드 방식이다.

// TLS 하이브리드 흐름



"TLS = 공개키로 전부 암호화"는 흔한 오해 — 실제 대량 데이터는 대칭키로 처리

자체 서명 인증서를 쓰기 때문에 클라이언트 쪽에서 `verify_none` 으로 검증을 생략한다 — 테스트 환경 한정이고, 프로덕션이라면 반드시 CA 인증서를 신뢰 목록에 등록해서 검증해야 한다.

3-2. ApmSession 독립 작성 — 왜 공용 Session을 건드리지 않았나

`GW2_CrossPlatformCore`엔 이미 `Session`이라는 공유 클래스가 있다. TLS를 붙일 때 이걸 확장하는 대신 **APM_Agent 전용의 독립된 ApmSession을 새로 작성했다.**

- **문제:** 공유 클래스에 TLS를 수술해서 넣으면, TLS를 안 쓰는 다른 소비자까지 영향을 받는다.
- **판단:** TLS 소비자가 하나뿐인 시점에 공용 클래스 책임을 넓히는 건 과설계(YAGNI).
- **해결:** Session은 완전 무수정, ApmSession이 프레이밍·암호화·재연결까지 전담.

3-3. 왜 TLS만으로 안 끝내고 페이로드를 한 번 더 암호화하나

TLS는 **네트워크 구간만** 보호한다. 로드밸런서/프록시를 거치면 그 지점에서 해제되기도 한다. 그래서 페이로드 자체를 애플리케이션 레벨에서 한 번 더 감싸서, TLS가 어딘가에서 해제되더라도 그 필드는 여전히 보호되게 설계했다.

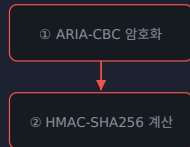
3-4. 처음 선택 — ARIA-256-CBC + HMAC-SHA256

ARIA는 KISA(한국인터넷진흥원)가 제정한 국내 표준 블록 암호다. "국가 표준 암호를 실무 수준으로 다뤄본 경험"이 포트폴리오 차별화 포인트가 될 거라고 판단해서 선택했다.

CBC 모드의 근본적 한계: CBC는 기밀성만 제공하고 무결성은 보장하지 않는다. 패딩 유효성이 새어나가면 **패딩 오라클 공격**으로 평문을 한 바이트씩 복원당할 수 있다.

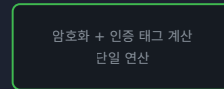
// Encrypt-then-MAC (CBC+HMAC) vs AEAD (GCM) - 이 프로젝트 최대 설계 포인트

ARIA-CBC + HMAC (개발자가 직접 조합)



△ 순서를 실수하면
패딩 오류를 재발
(복호화 전 MAC부터
검증해야 안전)

AES-256-GCM (AEAD, 라이브러리가 처리)



✓ 조합 순서를 실수할
방법 자체가 없음
태그 검증 실패 시
평문이 아예 안 나옴

"올바르게 구현하면 안전한 것"(CBC+HMAC) vs "잘못 구현할 방법이 없어서 안전한 것"(AEAD)

```
// SecurePayload::Open() - 순서가 핵심
// HMAC 검증을 먼저, 통과한 경우에만 ARIA 복호화(패딩 검증) 시도
// 변조된 암호문은 HMAC 단계에서 균일하게 거부 → 패딩 유효성 정보가 새어나갈 방법이 없음
```

실제 검증: 암호문을 변조한 뒤 `Open()` 을 호출하면 `HmacUtil::Verify()` 단계에서 먼저 예외가 발생함을 확인. 대칭키를 일부러 불일치시켜(HMAC 키만 일치·ARIA 키만 불일치) 같은 현상(HMAC 통과 → ARIA 복호화 단계에서 실패)을 재현해서 두 계층이 각각 올바르게 동작함을 실증했다.

3-5. AES-256-GCM으로 마이그레이션 - 왜 뒤집었나

두 구간(Agent↔Collector는 ARIA, Collector↔Console은 AES-GCM)을 **모두 AES-256-GCM으로 통일**했다.

1. **구조적 안전성:** 위 다이어그램처럼, CBC+HMAC의 조합 실수 여지 자체가 GCM에는 없다.
2. **단일 검증 경로:** `AesGcmCipher` / `AesGcmPayload` 는 GoogleTest 9개로 검증돼 있었지만 `AriaCipher` / `HmacUtil` / `SecurePayload` 는 자동 테스트가 없는 비대칭이 있었다. 하나로 합치면 테스트 9개가 두 구간 모두를 커버한다.

잃은 것과 보존 방법 — ARIA 코드(`AriaCipher` / `HmacUtil` / `SecurePayload`)는 삭제하지 않고 그대로 남김(실행 경로에서만 제외, "참고용" 주석 추가). 구현·검증 경험은 `Docs/ARIA_TO_AES_MIGRATION.md` 에 별도 정리.

// 수정 전

```
AriaCipher::Key encKey = LoadKeyFromHexFile("certs/aria.key");
HmacUtil::Key macKey = LoadKeyFromHexFile("certs/hmac.key");
...
std::make_unique<SecurePayload>(encKey, macKey)
```

// 수정 후

```
AesGcmCipher::Key agentCollectorKey = LoadKeyFromHexFile("certs/agent_collector_aes.key");
...
std::make_unique<AesGcmPayload>(agentCollectorKey)
```

세션 생성 로직, 프레임링 로직, 재연결 로직은 **단 한 줄도 안 바뀌었다**.

`IPayloadSealer` 인터페이스로 관심사를 분리해둔 덕분 — 마이그레이션 자체가 이 설계의 증거가 됐다.

3-6. 구간별 키 분리

같은 암호 방식(AES-256-GCM)을 쓰지만 키는 서로 다르다: Agent↔Collector는 `agent_collector_aes.key`, Collector↔Console은 `webserver_aes.key`. 한 구간의 키가 유출돼도 다른 구간은 안전하게 유지하기 위함이다.

3-7. Collector↔Console 관계의 진화

처음엔 두 프로세스가 **같은 장비의 SQLite 파일을 각자 다른 시점에 여는** 방식이었다(암 목적 전제). 이걸 Collector가 능동적으로 전송하는 명시적 네트워크 관계로 바꿨다 — 서로 다른 장비에 있어도 동작하도록.

`collector_config.json` 을 굳이 JSON으로 쓴 이유는 **차후 웹에서 이 설정을 편집할 가능성**을 열어두기 위해서다(웹과 자연스럽게 주고받을 수 있는 포맷).

Q. 왜 기존 Session 클래스를 확장하지 않고 ApmSession을 새로 만들었나요?

→ TLS가 필요한 소비자가 이 시점엔 APM_Agent 하나뿐이었습니다. 하나의 소비자를 위해 여러 소비자가 공유하는 클래스의 책임 범위를 미리 넓혀두는 건 YAGNI 원칙에 어긋난

다고 판단했고, 그래서 공유 Session은 완전히 무수정으로 두고 APM 전용 관심사는 전부 별도 클래스에 담았습니다.

Q. 왜 두 구간에 서로 다른 암호를 섞어 쓰다가 통일했나요?

→ Agent↔Collector는 KISA 표준 암호(ARIA)를 실무 수준으로 다뤄보는 게 목적이었고, Collector↔Console은 .NET과의 상호운용성 때문에 AES-GCM을 썼습니다. 나중에 되짚어보니 CBC+HMAC의 수동 조합은 구조적으로 실수 여지가 있고, 테스트도 AES-GCM에만 있는 비대칭이 있어서, GCM의 AEAD 구조적 안전성과 단일 검증 경로라는 이유로 통일했습니다.

Q. AEAD가 CBC+HMAC보다 왜 더 안전한가요?

→ CBC+HMAC은 두 개의 독립된 연산을 개발자가 특정 순서로 직접 조합해야 하고, 순서를 실수하면 패딩 오라클 공격면이 열립니다. GCM 같은 AEAD는 암호화와 인증을 라이브러리 내부에서 하나의 연산으로 처리해서, 조합 순서를 실수할 방법 자체가 없습니다.

Q. 패딩 오라클 공격이 정확히 뭔가요?

→ CBC 모드는 마지막 블록의 패딩(PKCS7) 유효성을 복호화 시 검증하는데, 공격자가 암호문을 한 바이트씩 조작해가며 '패딩이 유효했는지'를 서버 응답 차이로 구별할 수 있으면, 이 정보를 반복 질의해서 평문을 한 바이트씩 복원할 수 있는 공격입니다. Encrypt-then-MAC으로 감싸서 변조된 암호문을 패딩 검증 단계에 도달하기 전에 차단하면 이 공격면 자체가 사라집니다.

Q. IV/Nonce를 재사용하면 왜 안 되나요?

→ CBC에서 IV를 재사용하면 같은 평문 블록이 같은 암호문 블록을 만들어 패턴이 노출됩니다. GCM은 더 치명적인데, 같은 키로 Nonce를 재사용하면 인증 태그를 위조할 수 있어 기밀성과 무결성이 동시에 깨집니다. 그래서 IV/Nonce를 호출자가 지정할 방법 자체를 없애고(RAND_bytes()로 매번 내부 생성) 재사용을 코드 레벨에서 차단했습니다.

4. 통신 프로토콜

4-1. 메시지 프레임 — TCP는 스트림이라 경계가 없다

TCP는 **스트림 프로토콜**이라 "한 번의 read가 정확히 하나의 논리적 메시지"라는 보장이 없다.

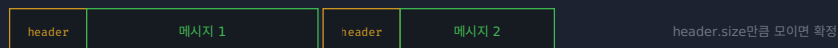
```
// TCP 원시 바이트 스트림 vs 프레임된 메시지
```

Before — 경계 없는 스트림

...bytes bytes bytes bytes bytes bytes bytes...

어디까지가 메시지 1개인지 알 수 없음

After — `PacketHeader{size,id}`로 프레임



```
void ApmSession::ProcessAccumulated()
{
    while (true)
    {
        size_t remaining = _accumulated.size() - processed;
        if (remaining < HEADER_SIZE) break;
        const PacketHeader* header = ...;
        if (remaining < header->size) break; // 아직 페이로드까지 다 안 모임
        // 완전한 패킷 하나 확정 → 복호화 → 콜백 호출
    }
}
```

4-2. Protobuf 리플렉션으로 패킷 ID 자동 결정

수동 배정 대신 `PacketType::descriptor()→index()` — `.proto` 파일 안 선언 순서를 그대로 ID로 쓴다. ID 충돌·오타가 **구조적으로 발생할 수 없다**.

4-3. PacketHandler — 타입 안전 디스패치

```
template<typename PacketType>
static void Register(TypedHandler<PacketType> handler)
```

```

{
    uint16 id = static_cast<uint16>(PacketType::descriptor()->index());
    Handlers()[id] = [handler](const String& payload)
    {
        PacketType pkt;
        if (!pkt.ParseFromString(payload)) { /* 에러 로그 */ return; }
        handler(pkt);    // 호출자는 이미 파싱된 강타입 메시지만 받음
    };
}

```

4-4. 크로스 언어 스키마 공유 – Metric.proto

C++과 .NET은 같은 .proto 파일에서 각자 코드젠한다(C++: `protoc --cpp_out`, .NET: `Grpc.Tools + Google.Protobuf`, 실제 gRPC 서비스는 미사용). 스키마를 손으로 복사하면 시간이 지나며 어긋나는데(스키마 이원화), 같은 원본에서 코드젠하기 때문에 이 위험이 구조적으로 없다.

4-5. ResilientSender – 재연결/큐잉 설계

Agent→Collector, Collector→Console 양쪽 모두에서 재사용되는 C++ 클래스.

```

void ResilientSender::EnqueueRaw(uint16 id, String payload)
{
    if (_queue.size() >= _maxQueueSize)
        _queue.pop_front();    // 큐가 가득 차면 가장 오래된 것부터 버림
    _queue.push_back(QueuedPacket{ id, std::move(payload) });
    if (_connected && !_sending) FlushNext();
}

```

- 큐가 가득 차면 **가장 오래된 것부터 버린다** — 최신 데이터가 더 중요하다는 가정.
- 재연결은 5초 타이머로 재시도. 전송 성공 시에만 `pop_front()` — 실패하면 큐에 남겨두고 재연결 시 그 항목부터 재시도.
- **스코프**: 메모리 큐만 구현 — 프로세스가 죽으면 큐도 사라짐(디스크 영속화는 범위 밖).

Q. TCP가 스트림이라는 게 정확히 무슨 뜻인가요?

→ TCP는 메시지 단위가 아니라 바이트 스트림을 전달합니다. 송신 측이 send()를 3번 호출해도 수신 측은 recv() 한 번에 다 받을 수도, 반대로 여러 번에 나눠 받을 수도 있습니다. 메시지의 경계가 어디인지는 애플리케이션이 직접 정의해야 하고, 이게 메시지 프레이밍이 필요한 이유입니다.

Q. 큐가 가득 차면 오래된 것부터 버리는 게 데이터 유실 아닌가요?

→ 맞습니다, 의도적인 트레이드오프입니다. 무한정 쌓아두면 에이전트 자체가 무거워지는 게 더 큰 문제라고 판단했고, 모니터링 데이터는 최신 상태가 가장 중요하다는 도메인 특성상 오래된 데이터부터 버리는 게 합리적이라고 봤습니다.

// Storage

5. 데이터 저장

5-1. IMetricStore – 컴파일 타임 저장소 선택 (C++)

SQLite와 TimescaleDB 둘 다 지원하고 싶은데, **안 쓰는 백엔드의 의존성이 결과물에 남으면 안 된다.** 런타임 분기 대신 인터페이스 + CMake 옵션으로 **컴파일 타임에 하나만 선택한다.**

```
// MetricStoreFactory.cpp
#ifdef APM_STORAGE_SQLITE
#include "SqliteMetricStore.h"
#elif defined(APM_STORAGE_TIMESCALEDB)
#include "TimescaleMetricStore.h"
#else
#error "APM_STORAGE_SQLITE 또는 APM_STORAGE_TIMESCALEDB 중 하나가 정의되어야 함"
#endif
```

검증: SQLite 선택 시 `ldd`로 libpq가 결과 바이너리에 전혀 링크되지 않음을 실측 확인.

5-2. .NET(APM_Console)의 런타임 선택과의 대비

```
public void RegisterServices(IServiceCollection services, IConfiguration configuration)
{
    var backend = configuration["Apm:StorageBackend"] ?? "SQLite";
    services.AddDbContext<ApmDbContext>(options =>
    {
        if (backend == "TimescaleDB") options.UseNpgsql(connectionString);
        else options.UseSqlite(connectionString);
    });
}
```

.NET은 두 EF Core 프로바이더를 둘 다 참조해줘도 빌드에 지장이 없다. C++처럼 안 쓰는 라이브러리가 결과물에 딸려오는 문제가 없어서, 설정 파일 값 하나로 런타임에 전환 가능하게 단순화했다 — 언어의 컴파일 모델 차이를 반영한 의도적 선택.

Q. 왜 C++은 컴파일 타임, .NET은 런타임으로 저장소를 고르나요?

→ C++은 링크 시점에 안 쓰는 라이브러리(예: libpq)까지 결과물에 딸려오는 걸 막으려면 컴파일 타임 분기가 필요합니다. 반면 .NET은 두 EF Core 프로바이더를 다 참조해도 배포 크기나 의존성 문제가 크지 않아서, 설정 파일 값 하나로 런타임에 유연하게 전환하는 쪽이 더 실용적이었습니다.

// Metrics Implementation

6. 수집 지표 구현 상세

`ResourceCollector` (C++, Linux 전용)가 `/proc` 파일시스템을 파싱해서 지표를 만든다.

6-1. CPU 사용률 - jiffies 델타

```
double ResourceCollector::ComputeCpuUsage()
{
    uint64 idleAll = idle + iowait;
```

```

uint64 totalAll = user + nice + system + idle + iowait + irq + softirq + steal;
if (_hasPrevSample && totalAll > _prevTotal)
{
    uint64 idleDelta = idleAll - _prevIdle;
    uint64 totalDelta = totalAll - _prevTotal;
    usage = 100.0 * (1.0 - (double)idleDelta / (double)totalDelta);
}
}

```

한 시점 값만으론 "지금 사용률"을 알 수 없다 — **두 시점 사이의 델타**를 봐야 한다.

`idle` 뿐 아니라 `iowait` (디스크 I/O 대기)도 "쉬고 있는 시간"으로 합산하는 게 표준 계산 방식이다.

6-2. 메모리 — MemFree가 아니라 MemAvailable

`MemFree`를 쓰면 안 된다 — Linux는 남은 메모리를 페이지 캐시로 적극 활용해서, `MemFree`만 보면 "메모리가 거의 다 찼다"는 착시가 생긴다. `MemAvailable`은 캐시처럼 즉시 회수 가능한 메모리까지 포함한 "실질적으로 새 프로세스가 쓸 수 있는 메모리" 추정치다.

6-3. 네트워크 대역폭 — 델타 + 루프백 제외

`/proc/net/dev`도 누적값이라 델타가 필요하다. `steady_clock`으로 실제 경과 시간을 측정해서 정확한 바이트/초를 계산하고, 인터페이스 이름이 환경마다 다르므로 `lo`만 제외하고 나머지는 전부 합산한다.

6-4. TCP 연결 품질 — 커널에서 직접 조회

```

getsockopt(fd, IPPROTO_TCP, TCP_INFO, &tcpInfo, &len);
// rtt, rttvar, retransmits, total_retrans, snd_cwnd 전부 커널이 이미 추적 중인 값

```

별도로 ping을 보내거나 직접 측정할 필요가 없다 — 커널의 TCP 스택이 흐름 제어를 위해 계속 추적하는 값을 그냥 꺼내 쓰는 것.

6-5. 디스크 사용량

`statvfs()` — 블록 개수 × 블록 크기로 바이트 환산하는 POSIX 표준 함수.

Q. 왜 MemFree가 아니라 MemAvailable을 쓰나요?

→ MemFree는 아무것도 안 쓰고 있는 순수 여유 메모리인데, Linux는 남은 메모리를 페이지 캐시로 적극 활용하기 때문에 MemFree만 보면 실제로는 여유가 있는데도 부족한 것처럼 보이는 착시가 생깁니다. MemAvailable은 캐시처럼 필요시 즉시 회수 가능한 부분까지 커널이 계산해서 제공하는 값이라 실제 체감 여유 메모리에 훨씬 가깝습니다.

Q. TCP 재전송률 같은 걸 왜 애플리케이션에서 직접 측정 안 하나요?

→ 커널의 TCP 스택은 흐름/혼잡 제어를 위해 RTT, 재전송 횟수 등을 이미 실시간으로 추적하고 있습니다. getsockopt(TCP_INFO)로 그냥 읽어오면 되는데, 애플리케이션에서 별도로 재측정하면 정확도도 떨어지고 불필요한 오버헤드만 생깁니다.

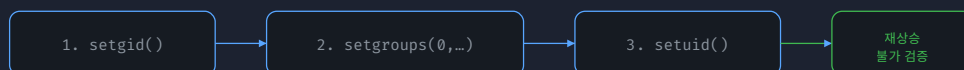
// Performance / Privilege

7. 성능/권한 설계

7-1. PrivilegeDrop - 최소 권한 원칙

침해당했을 때 공격자가 물려받는 권한 범위를 최소화하는 것이 목적.

// 권한 하향 순서 - 반드시 이 순서로



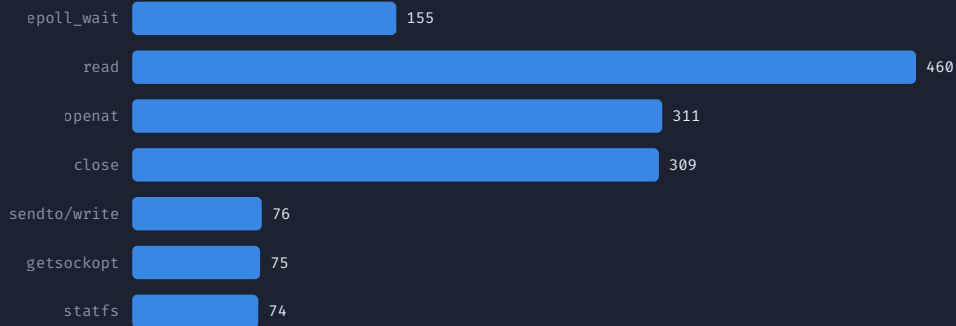
uid를 먼저 낮추면 root 권한이 사라져 이후 setgid/setgroups가 실패한다 — 순서가 핵심

재상승 불가 검증: 권한 하향 후 `setuid(0)`을 다시 시도해서 성공하면 오히려 문제다 — saved-set-uid가 여전히 0으로 남아있다는 뜻.

7-2. syscall 최소화 – 커널 모드 전환 비용

최소 권한(7-1)은 보안 관점, syscall 최소화는 성능 관점 — 완전히 독립된 두 개념.

// syscall 실측 (strace -c, Agent 5분 정상 동작 – 총 1,921회 / 19.4ms)



epoll_wait 블로킹 대기가 압도적 — openat/close(매 사이클 /proc 재오픈)가 유일한 반복 비용(전체의 32%)

다만 수집 주기를 훨씬 좁힐 계획이 생기면 파일 디스크립터 캐싱 최적화를 고려할 지점으로 남겨뒀다(알려진 최적화 여지를 스스로 인지하고 있다는 걸 보여주는 부분).

7-3. 리소스 사용량 실측

항목	값	비고
RSS	13.8MB 고정	20샘플(15초 간격, 5분) 전부 13,716~14,156KB — 누수 없음
CPU 평균	0.017%	utime+stime 합계 5 tick(HZ=100 기준)/300초

Q. 최소 권한과 syscall 최소화가 왜 다른 개념인가요?

→ 최소 권한은 프로세스가 침해당했을 때의 피해 범위(보안)를 줄이는 거고, syscall 최소화는 커널 모드 전환 비용(성능)을 줄이는 겁니다. 완전히 독립된 축이라 의도적으로 분리해서 설계·설명했습니다.

Q. 메모리 누수가 없다는 걸 어떻게 확인하나요?

→ 5분간 15초 간격으로 20번 RSS를 샘플링했는데 전부 13.8MB 근처에서 완전히 고정되어 있었습니다. 누수가 있었다면 지속적으로 증가하는 추세가 보였을 텐데 그런 패턴이 없었습니다. 다만 5분이라는 관측 시간의 한계상, 훨씬 긴 시간 스케일의 아주 느린 누수까지 배제한다고는 말할 수 없습니다.

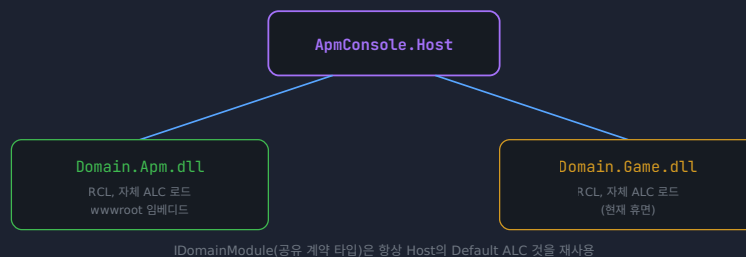
// APM_Console

8. APM_Console 아키텍처

8-1. 플러그인 아키텍처 – 왜 이렇게까지 나뉘었나

Apm 도메인과 Game 도메인을 한 프로젝트에 넣는 안(A안, 리플렉션 자동 등록)보다, 공수가 3~5배 더 들어도 **컴파일 타임에 강제되는 격리**가 있는 B안(도메인별 별도 프로젝트 + 런타임 로딩)을 선택했다.

// 플러그인 로딩 구조



8-2. DomainLoadContext – 타입 아이덴티티 함정 회피 (면접 단골 포인트)

같은 이름의 타입도 AssemblyLoadContext(ALC)가 다르면 완전히 다른 타입으로 취급된다. Host가 로드한 IDomainModule과 도메인 DLL이 별도 ALC에서 다시 로드한 같은 이름의 타입은 CLR 입장에서 별개 타입이라 is /캐스팅이 전부 실패한다.

```
protected override Assembly? Load(AssemblyName assemblyName)
{
    // 호스트(Default ALC)가 이미 로드해 둔 어셈블리는 그대로 재사용 -
    // 공유 계약 타입(IDomainModule 등)의 아이덴티티가 갈라지는 걸 막기 위함.
    var existing = AssemblyLoadContext.Default.Assemblies
        .FirstOrDefault(a => a.GetName().Name == assemblyName.Name);
    if (existing != null) return existing;

    // 호스트가 모르는 도메인 전용 패키지(EF Core, SQLite 등)는 도메인 폴더에서 정상 로드
    var assemblyPath = _resolver.ResolveAssemblyToPath(assemblyName);
    return assemblyPath != null ? LoadFromAssemblyPath(assemblyPath) : null;
}
```

8-3. SignalR 실시간 갱신 + 대시보드

// 폴링 vs push

폴링

변경 없어도 계속 요청(불강) — 딱 한 번만 실제 변경(조목), 나머지는 낭비 + 주기만큼 지연

SignalR push

변경된 순간에만 push, 지연 최소화

메트릭을 DB에 저장한 직후 곧바로 **MetricsHub** (`/apm/hub/metrics`)로 브로드캐스트한다. 대시보드는 그래프 5개(CPU/메모리/디스크 사용률, 네트워크 RX-TX, TCP RTT+RTT 분산) + 최근 메트릭 표(11컬럼), 최대 20개 유지.

Q. .NET 플러그인 아키텍처에서 타입 충돌이 왜 생기나요?

→ AssemblyLoadContext는 어셈블리 격리 단위입니다. 같은 이름·같은 코드의 타입이라도 서로 다른 ALC에서 각각 로드되면 CLR은 이것을 별개의 타입으로 취급합니다. 그래서 호스트와 플러그인이 별도 ALC에서 각자 로드한 같은 이름의 인터페이스 타입은 is 캐스팅이 실패하게 됩니다. 공유해야 하는 계약 타입은 항상 같은 ALC(보통 Default)에서 오도록 로딩 로직을 짜야 합니다.

Q. 왜 폴링 대신 SignalR을 썼나요?

→ 폴링은 변경이 없어도 계속 요청을 보내는 낭비가 있고 주기만큼의 지연이 항상 발생합니다. SignalR은 저장 직후 곧바로 push하기 때문에 불필요한 요청도 없고 지연도 최소화됩니다.

// Debugging

9. 발견한 버그 전체 (7건 상세)

"제안 단계에선 맞아 보였지만 실제 빌드/실행/렌더링에서 발견된" 버그 7건.

// Bug 1

Razor HTML 인코딩이 JS 타임스탬프를 깨뜨림

증상 대시보드 초기 렌더링에서 차트가 빈 그래프로 뜸.

원인 Razor의 `@` 표현식은 `<script>` 안이라도 항상 HTML 인코딩됨 — ISO 문자열의 "+"가 "+"로 깨져 `new Date()` 파싱 실패.

발견 실제 렌더링된 페이지를 `curl` 로 확인하다가 발견.

수정 ISO 문자열 대신 유닉스 밀리초(숫자)로 전달 — 숫자는 인코딩할 특수문자가 없음.

// Bug 2

TLS 핸드셰이크 모드 파라미터 누락

증상 Agent(client) 모드가 실질적으로 동작하지 않음(미정의 동작).

원인 생성자가 `mode` 를 안 받아 `_mode` 가 미초기화, `handshakeType` 도 계산만 하고 `server` 가 하드코딩된 채 남음.

발견 코드 리뷰(직접 대조)로 발견.

수정 `SessionMode mode` 파라미터 추가, 계산한 `handshakeType` 실제 전달.
`nm` 심볼 검증 + 실행 검증.

// Bug 3

ApmSession.h/.cpp가 0바이트 빈 파일로 방치

- 증상** "빌드 성공"이 곧 "코드가 있다"는 착각.
- 원인** 빈 .cpp도 정상 컴파일된다 — 빌드 성공만으론 못 잡음.
- 발견** `nm` 으로 오브젝트 파일에 심볼이 실제 존재하는지 확인하다가 발견.
- 수정** 내용 재작성 + 심볼 존재 재확인. "빌드 성공 ≠ 동작" 원칙이 생긴 계기.

// Bug 4

Storage 계층 3종 동시 결함

- 증상** 컴파일 자체가 안 됨.
- 원인** include 오타 + 빈 CMakeLists.txt + 헤더에 .cpp 내용 혼입, 3가지 동시 존재.
- 수정** 3건 모두 직접 수정 후 클린 빌드 재검증.

// Bug 5

ComputeNetworkUsage 중복 정의

- 증상** `redefinition` 컴파일 에러.
- 원인** 파일 끝에 빈 꺾이기 중복 정의 + `ComputeCpuUsage()` 통째로 누락.
- 수정** 중복 삭제 + 누락 함수 복원.

// Bug 6

TCP_INFO 연동 시 파일 누락 + 오타

- 증상** 링크/컴파일 에러.
- 원인** 관련 파일 4개 누락 + 타입명 오타(`TcpConnectonInfo`) + `.pb.h` 미재생성.
- 수정** 파일 복원, 오타 수정, 재생성. 클린 빌드 + `nm` + 실행 검증 (`rtt:32us,var:26us,retrans:0,cwnd:10` — 루프백 특성과 일치).

// Bug 7

ManifestEmbeddedFileProvider가 매니페스트를 못 찾음

증상 정적 리소스 404 — 예외가 조용히 삼켜져서 원인 파악 어려움.

원인 매니페스트 생성 MSBuild 타겟이 별도 NuGet 패키지 (`Microsoft.Extensions.FileProviders.Embedded`)에 있었음. `catch` 블록이 이 예외를 "정상 케이스"로 오인해서 삼킴.

발견 별도 콘솔 앱으로 `GetManifestResourceNames()` 직접 호출 + `curl` 로 최종 검증.

수정 패키지 참조 추가.

Q. 버그를 발견하는 본인만의 원칙이 있나요?

→ '빌드 성공은 동작을 증명하지 않는다'는 원칙입니다. 실제로 빈 파일이 정상 컴파일되는 걸 겪은 뒤로는, 심볼이 실제 존재하는지 nm으로 확인하고, 웹 쪽은 실제 렌더링된 결과를 curl로 직접 확인하는 걸 기본 절차로 삼았습니다.

Q. 예외가 조용히 삼켜져서 디버깅이 어려웠던 경험 있나요?

→ ManifestEmbeddedFileProvider 케이스가 정확히 그랬습니다. 특정 예외를 '정상적으로 있을 수 있는 케이스'로 오인해서 캐치 블록이 조용히 삼키고 있었는데 실제로는 진짜 설정 오류였습니다. 이후로는 catch 블록이 '정말 무해한 케이스만 걸러내는지'를 더 의심하게 됐습니다.

// Testing

10. 테스트 전략

구성	개수	검증 항목
AesGcmCipher(C++)	5	왕복 복원 / nonce 매번 다름 / 태그 변조 거부 / 암호문 변조 거부 / 다른 키 거부
AesGcmPayload(C++)	4	왕복 / 와이어 포맷 크기 / 변조 거부 / 인터페이스 경유 동작

ReadExactAsync(.NET) 4 전부 수신 / 분할 수신(ChunkedStream) / 조기 종료 / 길이 0

DecryptAndParse(.NET) 4 정상 왕복 / 태그·암호문 변조·잘못된 키 거부

테스트 가능하게 만들기 위한 리팩터링 — `MetricsReceiverService`의 `ReadExactAsync` / `DecryptAndParse` 는 원래 `private`였는데 `internal`로 넓히고 (`InternalsVisibleTo`), `DecryptAndParse` 는 인스턴스 필드 대신 **키를 파라미터로 받는 static 메서드**로 전환 — DI 없이도 순수 로직만 테스트 가능하게. "테스트 가능한 설계"는 의도적으로 리팩터링해야 얻어지는 것.

왜 이 두 개만 골랐나

1. 포트폴리오 특성상 시간 예산이 제한적 — "핵심 위험 지점 우선"이 합리적 우선순위.
2. Agent↔Collector도 AES-GCM으로 통일되면서, 이 9개가 **운영 경로 전체**를 커버하게 됨.
3. `ApmSession::ProcessAccumulated` 처럼 소켓에 강결합된 부분은 아직 자동 테스트 없음 — 인지하고 있는 갭.

Q. 왜 100% 테스트 커버리지를 목표로 하지 않았나요?

→ 시간 예산이 정해져 있었고, 모든 코드를 동일한 강도로 테스트하는 것보다 실수하면 조용히 실패하고 파급력이 큰 영역(암호화, 네트워크 프레이밍)을 우선하는 게 더 합리적인 투자라고 판단했습니다. 어디에 테스트가 없는지도 명확히 인지하고 있습니다.

Q. 테스트하기 어려운 코드를 어떻게 테스트 가능하게 만드나요?

→ 인스턴스 필드에 의존하던 로직을 파라미터를 받는 static 메서드로 바꿔서 DI 컨테이너 전체를 구성하지 않아도 순수 로직만 테스트할 수 있게 만듭니다. 접근 제한자는 최소한만 넓히고 `InternalsVisibleTo`로 테스트 프로젝트에만 노출해서 공개 API 표면은 유지합니다.

11. 예상 면접 질문 모음 (통합)

주제	질문	섹션
아키텍처	왜 이 코어를 다른 프로젝트에도 재사용했나	1절
아키텍처	Standalone Asio가 뭔가, Boost.Asio와 차이는	2절
아키텍처	크로스플랫폼이라면서 Windows는 왜 검증 안 했나	2절
보안	왜 Session을 확장 안 하고 ApmSession을 새로 만들었나	3절
보안	왜 두 암호를 섞어 쓰다 통일했나	3절
보안	AEAD가 CBC+HMAC보다 왜 더 안전한가	3절
보안	패딩 오라클 공격이 뭔가	3절
보안	IV/Nonce 재사용이 왜 위험한가	3절
프로토콜	TCP가 스트림이라는 게 무슨 뜻인가	4절
저장	왜 C++은 컴파일 타임, .NET은 런타임으로 저장소를 고르나	5절
수집	MemFree 대신 MemAvailable을 쓰는 이유	6절
성능	최소 권한과 syscall 최소화가 왜 다른가	7절
성능	메모리 누수 없다는 걸 어떻게 확신하나	7절
플러그인	.NET 플러그인 아키텍처에서 타입 충돌이 왜 생기나	8절
플러그인	왜 폴링 대신 SignalR인가	8절
디버깅	버그를 발견하는 본인만의 원칙이 있나	9절
테스트	왜 100% 커버리지를 목표로 안 했나	10절

포지셔닝(가장 중요) — "이 프로젝트를 만들게 된 계기가 뭔가요?" → "네트워크 프로
그래밍과 동시성 제어를 깊게 익히고 싶어서 직접 구현했다." 0절 네이밍 원칙 반드시
준수(게임 언급 금지).

// Glossary

12. 용어 미니 사전

용어	뜻
IOCP	Windows의 비동기 I/O 완료 통지(Completion Port) 메커니즘.
epoll	Linux의 비동기 I/O 준비 통지(readiness notification) 메커니즘.
AEAD	Authenticated Encryption with Associated Data — 암호화와 인증을 하나의 연산으로 처리하는 방식(예: AES-GCM).
Encrypt-then-MAC	암호화한 뒤 그 암호문에 대해 별도로 MAC을 계산·검증하는 패턴. 복호화 전에 MAC부터 검증해야 안전.
패딩 오라클 공격	CBC 모드에서 패딩 유효성 정보가 새어나가는 걸 악용해 평문을 복원하는 공격.
KISA	한국인터넷진흥원 — ARIA 등 국내 표준 암호를 제정.
jiffies	Linux 커널이 시간을 세는 단위. /proc/stat의 CPU 시간이 이 단위로 누적.
AssemblyLoadContext(ALC)	.NET에서 어셈블리를 격리해서 로드하는 단위. 플러그인 아키텍처의 핵심.
Razor Class Library(RCL)	.cshtml과 정적 파일을 어셈블리 안에 컴파일해 내장할 수 있게 해주는 .NET 프로젝트 타입.

// Live Demo

13. 라이브 데모 실행 순서

면접에서 실제 시연을 요청받을 경우를 대비한 순서(상세는 [Docs/WEBSERVER_TEST_SCENARIO.md](#)).

```
# 1) 인증서/키 생성(최초 1회)
cd APM_Agent
bash certs/generate_test_cert.sh
bash certs/generate_agent_collector_key.sh
bash certs/generate_webserver_key.sh
cd ../APM_Console && bash certs/generate_webserver_cert.sh

# 2) 빌드
cd APM_Agent && cmake -S . -B build -DAPM_STORAGE_BACKEND=SQLite && cmake --build build
cd ../APM_Console && dotnet build ApmConsole.sln

# 3) 실행 순서 (터미널 3개)
# 3-1) APM_Console 먼저
cd APM_Console && dotnet run --project src/ApmConsole.Host/ApmConsole.Host.csproj
# 3-2) Collector (반드시 APM_Agent/ 안에서)
cd APM_Agent && ./build/Collector
# 3-3) Agent
cd APM_Agent && ./build/Agent
```

브라우저로 [/apm/dashboard](#) 접속 — 5초마다 실시간 갱신을 보여줄 수 있다.

복원력 시연(선택): Collector를 강제 종료했다가 재시작 — Agent 로그에 `[ResilientSender] queued (size=N)`가 쌓이다가, 재시작 즉시 버퍼링분부터 순서대로 재전송되는 걸 보여줄 수 있다.

PROJECT_TECHNICAL_REVIEW.html · APM_Agent & APM_Console · 원본:
PROJECT_TECHNICAL_REVIEW.md